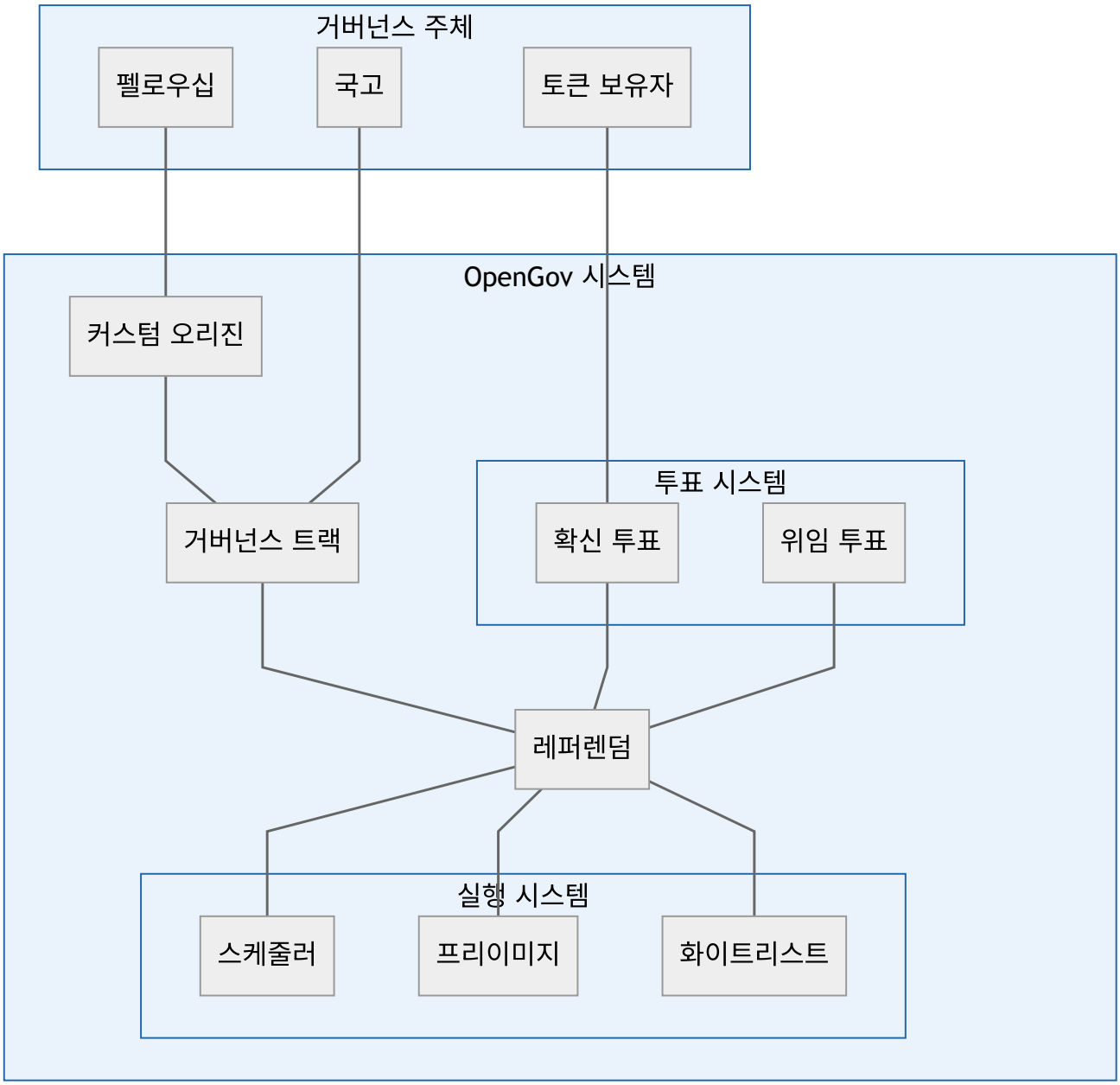
3.3 온체인 거버넌스 시스템 분석

개요

Polkadot의 온체인 거버넌스 시스템은 OpenGov(Governance v2)라고 불리는 혁신적인 거버넌스 모델을 채택하고 있습니다. 이 시스템은 기존의 Council과 Technical Committee 기반 거버넌스에서 발전하여, 더욱 민주적이고 효율적인 의사결정 구조를 제공합니다. 이 문서에서는 OpenGov 시스템의 구조, Referenda 팔렛, 다양한 거버넌스 트랙, 그리고 실제 참여 방법을 상세히 분석합니다.

OpenGov 시스템 아키텍처

거버넌스 시스템 개요



주요 컴포넌트

1. 커스텀 오리진 (Custom Origins)

OpenGov 시스템의 핵심은 다양한 권한 수준을 가진 커스텀 오리진입니다:

```
#[derive(PartialEq, Eq, Clone, MaxEncodedLen, Encode, Decode, TypeInfo, RuntimeD

#[pallet::origin]

pub  enum Origin {

/// 스테이킹 관리 권한
```

```
StakingAdmin,

/// 국고 지출 권한 (무제한)

Treasurer,

/// 펠로우십 관리 권한

FellowshipAdmin,

/// 일반 관리 권한

GeneralAdmin,

/// 경매 관리 권한

AuctionAdmin,

/// 리스 관리 권한

LeaseAdmin,

/// 레퍼렌덤 취소 권한

ReferendumCanceller,

/// 레퍼렌덤 강제 종료 권한

ReferendumKiller,

/// 소액 팁 권한 (250 WND)

SmallTipper,

/// 대액 팁 권한 (1,000 WND)

BigTipper,

/// 소액 지출 권한 (10,000 WND)

SmallSpender,

/// 중액 지출 권한 (100,000 WND)

MediumSpender,
```

```
/// 대액 지출 권한 (1,000,000 WND)
BigSpender,
/// 화이트리스트 호출 권한
WhitelistedCaller,
/// 펠로우십 등급별 권한 (1단~9단)
Fellowship1Dan,
Fellowship2Dan,
// ... Fellowship9Dan까지
}
```

2. 거버넌스 트랙 (Governance Tracks)

각 오리진은 특정 트랙에 매핑되며, 트랙별로 다른 투표 매개변수를 가집니다:

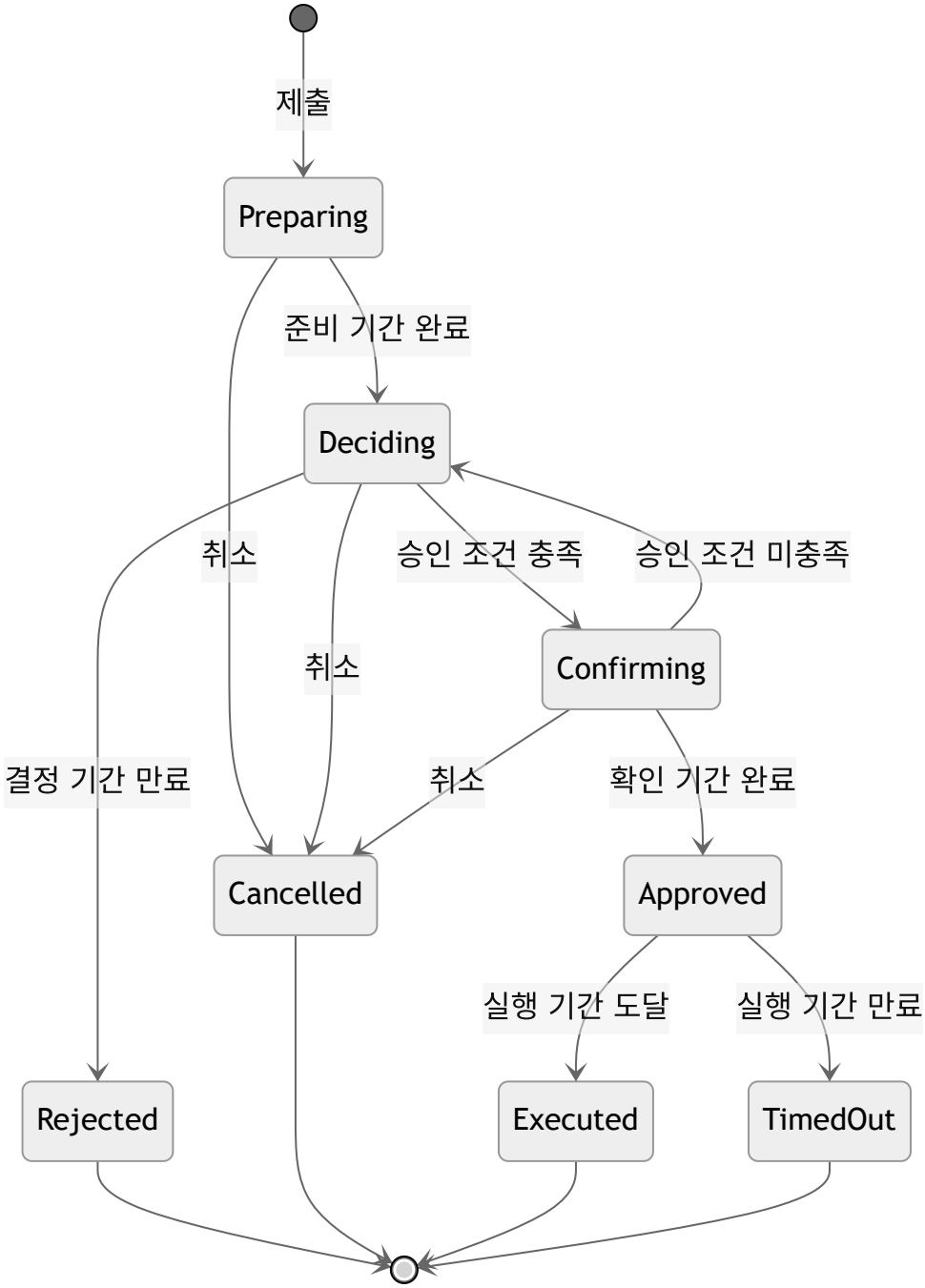
```
pub  const TRACKS_DATA: [pallet_referenda::Track<u16, Balance, BlockNumber>; 15]
// Root 트랙 (ID: 0) - 최고 권한
pallet_referenda::Track {
id: 0,
info: pallet_referenda::TrackInfo {
name: s("root"),
max_deciding: 1, // 동시 결정 가능한 레퍼렌덤 수
decision_deposit: 100 * GRAND, // 결정 보증금 (100,000 WND)
prepare_period: 8 * MINUTES, // 준비 기간
decision_period: 20 * MINUTES, // 결정 기간
```

```
confirm_period: 12 * MINUTES, // 확인 기간
min_enactment_period: 5 * MINUTES, // 최소 실행 기간
min_approval: APP_ROOT, // 최소 승인 곡선
min_support: SUP_ROOT, // 최소 지지 곡선
},
},
// 화이트리스트 호출자 트랙 (ID: 1)
pallet_referenda::Track {
id: 1,
info: pallet_referenda::TrackInfo {
name: s("whitelisted_caller"),
max_deciding: 100,
decision_deposit: 10 * GRAND, // 10,000 WND
prepare_period: 6 * MINUTES,
decision_period: 20 * MINUTES,
confirm_period: 4 * MINUTES,
min_enactment_period: 3 * MINUTES,
min_approval: APP_WHITELISTED_CALLER,
min_support: SUP_WHITELISTED_CALLER,
},
},
// 스테이킹 관리자 트랙 (ID: 10)
pallet_referenda::Track {
```

```
id: 10,
info: pallet_referenda::TrackInfo {
name: s("staking_admin"),
max_deciding: 10,
decision_deposit: 5 * GRAND, // 5,000 WND
prepare_period: 8 * MINUTES,
decision_period: 20 * MINUTES,
confirm_period: 8 * MINUTES,
min_enactment_period: 3 * MINUTES,
min_approval: APP_STAKING_ADMIN,
min_support: SUP_STAKING_ADMIN,
},
},
// ... 기타 트랙들
];
```

레퍼렌덤 시스템 (Referenda Pallet)

레퍼렌덤 라이프사이클



레퍼렌덤 매개변수

1. 시간 매개변수

```
parameter_types! {
pub const AlarmInterval: BlockNumber = 1; // 알람 간격 (1블록)
pub const SubmissionDeposit: Balance = 1 * 3 * CENTS; // 제출 보증금
pub const UndecidingTimeout: BlockNumber = 14 * DAYS; // 미결정 타임아웃
```

```
pub  const VoteLockingPeriod: BlockNumber = 7 * DAYS; // 투표 잠금 기간

}
```

2. 승인 및 지지 곡선

거버넌스 트랙별로 다른 승인(Approval)과 지지(Support) 곡선을 사용합니다:

```
// Root 트랙 곡선

const APP_ROOT: Curve = Curve::make_reciprocal(4, 28, percent(80), percent(50),

const SUP_ROOT: Curve = Curve::make_linear(28, 28, percent(0), percent(50));



// 스테이킹 관리자 트랙 곡선

const APP_STAKING_ADMIN: Curve = Curve::make_linear(17, 28, percent(50), percent

const SUP_STAKING_ADMIN: Curve = Curve::make_reciprocal(12, 28, percent(1), perc



// 국고 지출 트랙 곡선

const APP_TREASURER: Curve = Curve::make_reciprocal(4, 28, percent(80), percent(

const SUP_TREASURER: Curve = Curve::make_linear(28, 28, percent(0), percent(50))
```

곡선 해석:

- **Approval (승인)**: 찬성 투표 비율이 시간에 따라 어떻게 변해야 하는지
- **Support (지지)**: 전체 토큰 대비 투표 참여 비율이 어떻게 변해야 하는지

확신 투표 시스템 (Conviction Voting)

확신 투표 메커니즘

확신 투표는 투표자가 자신의 확신 정도에 따라 투표 가중치를 조절할 수 있는 시스템입니다:

```
pub enum Conviction {
None, // 1x 가중치, 잠금 없음
Locked1x, // 1x 가중치, 1 잠금 기간
Locked2x, // 2x 가중치, 2 잠금 기간
Locked3x, // 3x 가중치, 4 잠금 기간
Locked4x, // 4x 가중치, 8 잠금 기간
Locked5x, // 5x 가중치, 16 잠금 기간
Locked6x, // 6x 가중치, 32 잠금 기간
}
```

투표 가중치 계산

```
// 투표 가중치 = 토큰 양 × 확신 배수
fn vote_weight(balance: Balance, conviction: Conviction) -> Balance {
match conviction {
Conviction::None => balance,
Conviction::Locked1x => balance,
Conviction::Locked2x => balance * 2,
Conviction::Locked3x => balance * 3,
Conviction::Locked4x => balance * 4,
```

```
Conviction::Locked5x => balance * 5,

Conviction::Locked6x => balance * 6,

}

}



// 잠금 기간 = 기본 잠금 기간 × 2^(확신 레벨 - 1)

fn  lock_period(conviction: Conviction, base_period: BlockNumber) -> BlockNumber

match  conviction {

Conviction::None => 0,

Conviction::Locked1x => base_period,

Conviction::Locked2x => base_period * 2,

Conviction::Locked3x => base_period * 4,

Conviction::Locked4x => base_period * 8,

Conviction::Locked5x => base_period * 16,

Conviction::Locked6x => base_period * 32,

}

}
```

거버넌스 참여 방법

1. 레퍼렌덤 제출

기본 제출 과정

```
// Polkadot.js API를 사용한 레퍼렌덤 제출
const api = await ApiPromise.create({ provider: wsProvider });

// 1. 프리이미지 제출
const proposal = api.tx.system.remark("Hello Polkadot!");
const preimageHash = proposal.method.hash;

await api.tx.preimage.notePreimage(proposal.method.toHex()).signAndSend(account

// 2. 레퍼렌덤 제출
const origin = { system: "Root" }; // 또는 다른 커스텀 오리진
await api.tx.referenda.submit(
origin,
{ Lookup: { hash: preimageHash, len: proposal.method.encodedLength } },
{ After: 100 } // 100블록 후 실행
).signAndSend(account);
```

트랙별 제출 예제

```
// 스테이킹 관리자 트랙으로 제출
const stakingOrigin = { Origins: "StakingAdmin" };
await api.tx.referenda.submit(
stakingOrigin,
```

```
proposalHash,
enactmentMoment
).signAndSend(account);

// 국고 지출 트랙으로 제출
const treasuryOrigin = { Origins: "Treasurer" };
await api.tx.referenda.submit(
treasuryOrigin,
proposalHash,
enactmentMoment
).signAndSend(account);
```

2. 투표 참여

직접 투표

```
// 확신 투표로 찬성
await api.tx.convictionVoting.vote(
referendumIndex,
{
Standard: {
vote: {
aye: true,
conviction: "Locked2x" // 2배 가중치, 2 잠금 기간
```

```
},
balance: "1000000000000" // 1 WND (12 decimals)
}
}
).signAndSend(account);

// 분할 투표 (일부 찬성, 일부 반대)
await api.tx.convictionVoting.vote(
referendumIndex,
{
Split: {
aye: "600000000000", // 0.6 WND 찬성
nay: "400000000000" // 0.4 WND 반대
}
}
).signAndSend(account);
```

위임 투표

```
// 특정 트랙에 대한 투표 위임
await api.tx.convictionVoting.delegate(
trackId,
delegateAccount,
```

```
"Locked3x", // 확신 레벨
"5000000000000"  // 위임할 토큰 양 (5 WND)
).signAndSend(account);


// 위임 해제
await  api.tx.convictionVoting.undelegate(trackId).signAndSend(account);
```

3. 레퍼렌덤 관리

레퍼렌덤 취소

```
// 레퍼렌덤 취소 (ReferendumCanceller 권한 필요)
await  api.tx.referenda.cancel(referendumIndex).signAndSend(account);


// 레퍼렌덤 강제 종료 (ReferendumKiller 권한 필요)
await  api.tx.referenda.kill(referendumIndex).signAndSend(account);
```

보증금 환불

```
// 제출 보증금 환불
await  api.tx.referenda.refundSubmissionDeposit(referendumIndex).signAndSend(acc


// 결정 보증금 환불
await  api.tx.referenda.refundDecisionDeposit(referendumIndex).signAndSend(accou
```

실제 거버넌스 사례 연구

1. 런타임 업그레이드 제안

```
// 런타임 업그레이드 제안 예제
const newRuntimeWasm = "0x...";  // 새로운 런타임 WASM 코드

// 1. 프리이미지 제출
const upgradeCall = api.tx.system.setCode(newRuntimeWasm);
await api.tx.preimage.notePreimage(upgradeCall.method.toHex()).signAndSend(acco

// 2. Root 트랙으로 제출
await api.tx.referenda.submit(
{ system: "Root" },
{ Lookup: { hash: upgradeCall.method.hash, len: upgradeCall.method.encodedLengt
{ After: 28800 } // 2일 후 실행 (블록당 6초 기준)
).signAndSend(account);
```

2. 국고 지출 제안

```
// 중액 국고 지출 제안 (100,000 WND)
const beneficiary = "5GrwvaEF5zXb26Fz9rcQpDWS57CtERHpNehXCPcNoHGKutQY";
const amount = "100000000000000000";  // 100,000 WND
```

```
const treasurySpendCall = api.tx.treasury.spend(amount, beneficiary);
await api.tx.preimage.notePreimage(treasurySpendCall.method.toHex()).signAndSen

// MediumSpender 트랙으로 제출
await api.tx.referenda.submit(
{ Origins: "MediumSpender" },
{ Lookup: { hash: treasurySpendCall.method.hash, len: treasurySpendCall.method.
{ After: 14400 } // 1일 후 실행
).signAndSend(account);
```

3. 스테이킹 매개변수 변경

```
// 스테이킹 매개변수 변경 제안
const newSessionsPerEra = 8; // 에라당 세션 수를 8로 변경

const stakingCall = api.tx.staking.setSessionsPerEra(newSessionsPerEra);
await api.tx.preimage.notePreimage(stakingCall.method.toHex()).signAndSend(acco

// StakingAdmin 트랙으로 제출
await api.tx.referenda.submit(
{ Origins: "StakingAdmin" },
{ Lookup: { hash: stakingCall.method.hash, len: stakingCall.method.encodedLengt
```

```
{ After: 7200 } // 12시간 후 실행
).signAndSend(account);
```

거버넌스 모니터링 및 분석

1. 레퍼렌덤 상태 조회

```
// 활성 레퍼렌덤 목록 조회
const referendumCount = await api.query.referenda.referendumCount();
console.log(`총 레퍼렌덤 수: ${referendumCount}`);

// 특정 레퍼렌덤 정보 조회
const referendumInfo = await api.query.referenda.referendumInfoFor(referendu
if (referendumInfo.isSome) {
const info = referendumInfo.unwrap();
console.log(`레퍼렌덤 ${referendumIndex} 상태:`, info.toHuman());
}

// 트랙별 진행 중인 레퍼렌덤 수 조회
const decidingCount = await api.query.referenda.decidingCount(trackId);
console.log(`트랙 ${trackId} 진행 중인 레퍼렌덤 수: ${decidingCount}`);
```

2. 투표 현황 분석

```
// 특정 계정의 투표 현황 조회
const votingFor = await api.query.convictionVoting.votingFor(account, track
console.log(`계정 ${account}의 트랙 ${trackId} 투표 현황:`, votingFor.toHuman());


// 레퍼렌덤 투표 집계 조회
const tally = await api.derive.referenda.referendumInfo(referendumIndex);
if (tally) {
console.log(`레퍼렌덤 ${referendumIndex} 투표 집계:`, {
ayes: tally.ayes.toString(),
nays: tally.nays.toString(),
support: tally.support.toString(),
approval: tally.approval.toString()
});
}
```

3. 거버넌스 메트릭

```
// 거버넌스 활동 메트릭 수집
async function collectGovernanceMetrics() {
const metrics = {
totalReferenda: await api.query.referenda.referendumCount(),
activeReferenda: 0,
```

```
totalVotes: 0,
totalStaked: 0
};
// 각 트랙별 활성 레퍼렌덤 수 계산
for (let trackId = 0; trackId < 35; trackId++) {
const deciding = await api.query.referenda.decidingCount(trackId);
metrics.activeReferenda += deciding.toNumber();
}
return metrics;
}
```

거버넌스 베스트 프랙티스

1. 제안서 작성 가이드라인

제안서 구조:

1. **제목**: 명확하고 간결한 제안 제목
2. **요약**: 제안의 핵심 내용 요약
3. **배경**: 제안이 필요한 이유와 배경
4. **상세 내용**: 구체적인 변경 사항
5. **영향 분석**: 예상되는 영향과 부작용
6. **실행 계획**: 단계별 실행 방법
7. **예산**: 필요한 자원과 비용

2. 투표 전략

투표자를 위한 가이드라인:

- 제안서를 충분히 검토하고 이해하기
- 확신 레벨을 신중하게 선택하기
- 장기적 관점에서 네트워크 이익 고려하기
- 위임 투표 시 신뢰할 수 있는 대표 선택하기

3. 보안 고려사항

제안자를 위한 보안 체크리스트:

- [ ] 프리이미지 해시 검증
- [ ] 실행 시점 적절성 확인
- [ ] 권한 레벨 적정성 검토
- [ ] 부작용 및 리스크 분석
- [ ] 커뮤니티 피드백 수렴

미래 거버넌스 발전 방향

1. 예상되는 개선사항

- **동적 트랙 조정**: 네트워크 상황에 따른 트랙 매개변수 자동 조정
- **AI 기반 제안 분석**: 제안서의 영향도 자동 분석
- **크로스체인 거버넌스**: 파라체인 간 거버넌스 연동
- **유동 민주주의**: 더욱 유연한 위임 시스템

2. 거버넌스 참여 확대 방안

- **교육 프로그램**: 거버넌스 참여 교육 강화

- **인센티브 시스템**: 적극적 참여에 대한 보상
- **사용자 경험 개선**: 더 직관적인 거버넌스 인터페이스
- **다국어 지원**: 글로벌 참여 확대

이 문서는 Polkadot의 OpenGov 시스템에 대한 포괄적인 이해를 제공하며, 실제 거버넌스 참여를 위한 실용적인 가이드를 포함합니다. 거버넌스는 네트워크의 미래를 결정하는 중요한 과정이므로, 모든 참여자가 책임감을 가지고 참여하는 것이 중요합니다.